

File Conversion System

Computer Network -- 2nd Year CN Project Report

Student 1 Name:- Chiranthan Shankar

Student 2 Name:- Hamza Shabbir Sahapurwala

Student 1 SRN:- PES2UG24CS137

Student 2 SRN:- PES2UG24CS177

1. What Does It Do?

This project is a client-server system that lets a client send a Word document (.docx) over a network to a server, which converts it to a PDF and sends it back. There are 4 Python files that each handle a different part of this:

- server.py — runs on the server, listens for connections and handles converting files
- client.py — the client that sends files and receives results
- conversion.py — does the actual .docx to .pdf conversion
- performance_graph.py — reads a CSV of recorded results and draws a graph

2. server.py

This is the main server file. It opens a TCP socket on port 5001 and waits for clients to connect.

Code:

```
import socket
import os
import threading
import time
from queue import Queue
import conversion
```

```
HOST = "0.0.0.0"
```

```
PORT = 5001
```

```
BUFFER_SIZE = 4096
```

```
ALLOWED_EXTENSIONS = [".docx", ".pdf", ".txt", ".doc"]
```

```

# Job queue
job_queue = Queue()

def receive_exact(conn, size):
    data = b""
    while len(data) < size:
        packet = conn.recv(size - len(data))
        if not packet:
            return None
        data += packet
    return data

def receive_file(conn):
    raw = receive_exact(conn, 16)
    if not raw:
        return None

    filename_size = int(raw.decode().strip())
    filename = receive_exact(conn, filename_size).decode()

    file_size = int(receive_exact(conn, 16).decode().strip())

    if not any(filename.endswith(ext) for ext in ALLOWED_EXTENSIONS):
        print("✗ File type not allowed")
        return None

    with open(filename, "wb") as f:
        bytes_received = 0
        while bytes_received < file_size:
            data = conn.recv(min(BUFFER_SIZE, file_size - bytes_received))
            if not data:
                return None
            f.write(data)
            bytes_received += len(data)

    print("📁 File received:", filename)
    return os.path.abspath(filename)

def send_file(conn, filepath):
    filename = os.path.basename(filepath)
    file_size = os.path.getsize(filepath)

```

```

conn.sendall(f"{len(filename):<16}".encode())
conn.sendall(filename.encode())
conn.sendall(f"{file_size:<16}".encode())

with open(filepath, "rb") as f:
    while True:
        data = f.read(BUFFER_SIZE)
        if not data:
            break
        conn.sendall(data)

print("📁 Sent:", filename)

# ----- WORKER THREAD -----

def worker(worker_id): # they are made first and they keep waiting for a job to
    take and complete(FSFC)
    while True:
        conn, filepath = job_queue.get() # if no jobs are present inside queue,
        the code will not go forward

        try:
            print(f"🌀 Worker {worker_id} processing:", filepath)

            # simulate heavy processing (demo)
            time.sleep(5)

            converted_file = conversion.start(filepath)

            if converted_file and os.path.exists(converted_file):
                conn.sendall("SUCCESS".ljust(16).encode())
                send_file(conn, converted_file)
            else:
                conn.sendall("FAILED".ljust(16).encode())

        except Exception as e:
            print("Worker error:", e)

        conn.close()
        job_queue.task_done()

# ----- CLIENT HANDLER -----

```

```

def handle_client(conn, addr):
    print("🔗 Connected:", addr)

    try:
        command = receive_exact(conn, 16).decode().strip()

        if command == "UPLOAD":

            filepath = receive_file(conn) # calls the binary file transfer file

            if filepath:
                print("📁 Job added to queue")
                job_queue.put((conn, filepath)) # job added inside the job queue
            else:
                conn.sendall("FAILED".ljust(16).encode())
                conn.close()

        else:
            print("❌ Unknown command")
            conn.close()

    except Exception as e:
        print("Server error:", e)
        conn.close()

# ----- SERVER -----

def start_server():

    # Start multiple workers (parallel job processing)
    for i in range(3):
        threading.Thread(target=worker, args=(i+1,), daemon=True).start() # makes
        worker threads and keep them waiting at the queue for a job to complete

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

        s.bind((HOST, PORT))
        s.listen()

        print("👂 Server listening on port", PORT)

        while True:
            conn, addr = s.accept()

```

```

        # concurrent client handling
        thread = threading.Thread(target=handle_client, args=(conn, addr))
        thread.start()

if __name__ == "__main__":
    start_server()

```

How it works

- When a client connects, it spins up a new thread to handle that client (so multiple clients can connect at once).
- The client sends an UPLOAD command, then the file. The server saves the file and adds it to a job queue.
- There are 3 worker threads constantly watching the queue. When a job appears, a worker picks it up and converts the file.
- After conversion, the server sends back either SUCCESS + the PDF, or FAILED.

Things worth noting

- The `receive_exact()` function is important — TCP doesn't guarantee you get all the data in one `recv()` call, so this loops until it has exactly the right number of bytes.
- There's a `time.sleep(5)` in the worker to fake slow processing — that's just for demo purposes.
- Files are saved with whatever name the client sends, which could be a problem (e.g. someone sends a file named `../../etc/passwd`).

```

PS C:\Users\Chiru\Desktop\VSPrograms\WSL_programs> & C:\Users\Chiru\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:\Users\Chiru\Desktop\VSPrograms\WSL_programs\CN_LAB\server.py"
Created: C:\Users\Chiru\Desktop\VSPrograms\WSL_programs\t2.pdf
Sent: t2.pdf
Connected: ('10.1.16.151', 61202)
File received: t1.docx
Job added to queue
Worker 1 processing: C:\Users\Chiru\Desktop\VSPrograms\WSL_programs\t1.docx
Conversion successful!
Created: C:\Users\Chiru\Desktop\VSPrograms\WSL_programs\t1.pdf
Sent: t1.pdf
Connected: ('10.1.16.151', 61205)
File received: t2.docx
Job added to queue
Worker 2 processing: C:\Users\Chiru\Desktop\VSPrograms\WSL_programs\t2.docx
Conversion successful!
Created: C:\Users\Chiru\Desktop\VSPrograms\WSL_programs\t2.pdf
Sent: t2.pdf

```

3. Client File

The client file is `client.py`. It asks you to type in a file path, sends it to the server, and waits for the converted PDF.

Code:

```
import socket
import os
import time

SERVER_IP = "10.1.19.100" # change if needed
PORT = 5001
BUFFER_SIZE = 4096 # 4KB packet/buffer size

ALLOWED_EXTENSIONS = [".docx", ".pdf", ".txt", ".doc"]

def receive_exact(sock, size):
    data = b""
    while len(data) < size:
        packet = sock.recv(size - len(data))
        if not packet:
            return None
        data += packet
    return data

def send_file(sock, filepath):

    if not os.path.exists(filepath):
        print("✗ File does not exist")
        return False

    filename = os.path.basename(filepath)

    if not any(filename.endswith(ext) for ext in ALLOWED_EXTENSIONS):
        print("✗ File type not allowed")
        return False

    file_size = os.path.getsize(filepath)

    sock.sendall(f"{len(filename):<16}".encode())
    sock.sendall(filename.encode())
    sock.sendall(f"{file_size:<16}".encode())

    with open(filepath, "rb") as f:
        while True:
            data = f.read(BUFFER_SIZE)
            if not data:
```

```

        break
    sock.sendall(data)

print("📁 Uploaded:", filename)
return True

def receive_file(sock):

    filename_size = int(receive_exact(sock, 16).decode().strip())
    filename = receive_exact(sock, filename_size).decode()
    file_size = int(receive_exact(sock, 16).decode().strip())

    save_name = "client_received_" + filename

    with open(save_name, "wb") as f:
        bytes_received = 0
        while bytes_received < file_size:
            data = sock.recv(min(BUFFER_SIZE, file_size - bytes_received))
            if not data:
                return
            f.write(data)
            bytes_received += len(data)

    print("📁 Downloaded converted file:", filename)

def main():

    filepath = input("Enter file path to upload: ")

    start_time = time.time()

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

        s.connect((SERVER_IP, PORT))

        s.sendall("UPLOAD".ljust(16).encode())

        if send_file(s, filepath):

            print("⌚ Waiting for server processing...")

            status = receive_exact(s, 16).decode().strip()

```

```

        if status == "SUCCESS":
            receive_file(s)
        else:
            print("❌ Conversion failed")

end_time = time.time()

file_size = os.path.getsize(filepath)
time_taken = end_time - start_time

print("📊 File size:", file_size, "bytes")
print("📊 Time taken:", time_taken, "seconds")

# Save performance data
with open("performance.csv", "a") as f:
    f.write(f"{file_size},{time_taken}\n")

if __name__ == "__main__":
    while True:
        main()

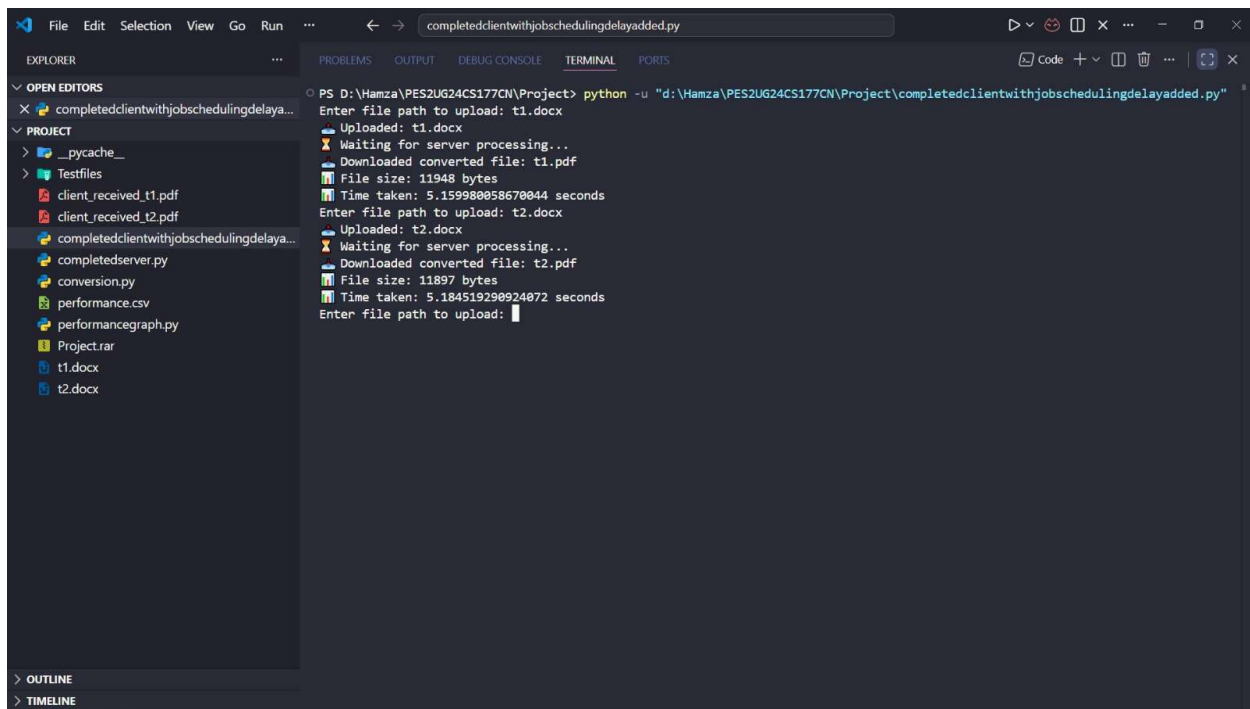
```

How it works

- Connects to the server IP (hardcoded as 172.28.242.85) on port 5001.
- Sends UPLOAD then the file using a simple custom protocol — filename length, filename, file size, then the actual file bytes.
- Waits for the server to respond with SUCCESS or FAILED.
- If successful, receives the PDF and saves it as client_received_<filename>.pdf.
- Records the file size and how long it took into performance.csv.
- Loops forever so you can keep sending files without restarting.

Things worth noting

- The server IP is hardcoded — you'd need to change it manually if the server moves.
- No timeout is set, so if the server crashes mid-transfer the client will just hang forever.
- The performance.csv data is used by performance_graph.py to plot results.



4. conversion.py

This is the module that does the actual file conversion. The server calls `conversion.start()` from the worker thread.

Code:

```
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import inch
from docx import Document
import os
```

```
def docx_to_pdf(docx_file, pdf_file):
    # Load DOCX
    document = Document(docx_file)

    # Create PDF
    pdf = SimpleDocTemplate(pdf_file, pagesize=A4)
    elements = []
    styles = getSampleStyleSheet()
    normal_style = styles["Normal"]

    # Add paragraphs to PDF
```

```

for para in document.paragraphs:
    text = para.text.strip()
    if text:
        elements.append(Paragraph(text.replace("\n", "<br/>"), normal_style))
        elements.append(Spacer(1, 0.2 * inch))

pdf.build(elements)

print("✅ Conversion successful!")
print(f"📄 Created: {pdf_file}")

def start(filepath):
    if not os.path.exists(filepath):
        print("❌ File does not exist.")
        return None

    if not filepath.lower().endswith(".docx"):
        print("❌ Not a DOCX file.")
        return None

    # Generate output filename safely
    base_name = os.path.splitext(filepath)[0]
    converted_file = base_name + ".pdf"

    try:
        docx_to_pdf(filepath, converted_file)
    except Exception as e:
        print("❌ Conversion failed:", e)
        return None

    return converted_file

```

How it works

- Opens the .docx file with the python-docx library.
- Loops through every paragraph and adds it to a ReportLab PDF document.
- Saves the PDF with the same name but .pdf extension.
- Returns the path to the new PDF (or None if it failed).

Limitations

- Only copies plain text — images, tables, and formatting from the Word doc are lost.

- For a real converter you'd want to use something like LibreOffice in headless mode instead, which preserves much more of the original formatting.

5. performance_graph.py

A simple standalone script to visualise the performance data collected by the client.

Code:

```
import csv
import matplotlib.pyplot as plt

file_sizes = []
times = []

# read csv data
with open("performance.csv", "r") as f:
    reader = csv.reader(f)

    for row in reader:
        size = float(row[0])
        time = float(row[1])

        file_sizes.append(size)
        times.append(time)

# convert bytes → KB (for nicer graph)
file_sizes_kb = [s/1024 for s in file_sizes]

plt.figure()

plt.plot(file_sizes_kb, times, marker='o')

plt.title("Performance vs File Size Analysis")
plt.xlabel("File Size (KB)")
plt.ylabel("Transfer Time (seconds)")

plt.grid(True)

plt.show()
```

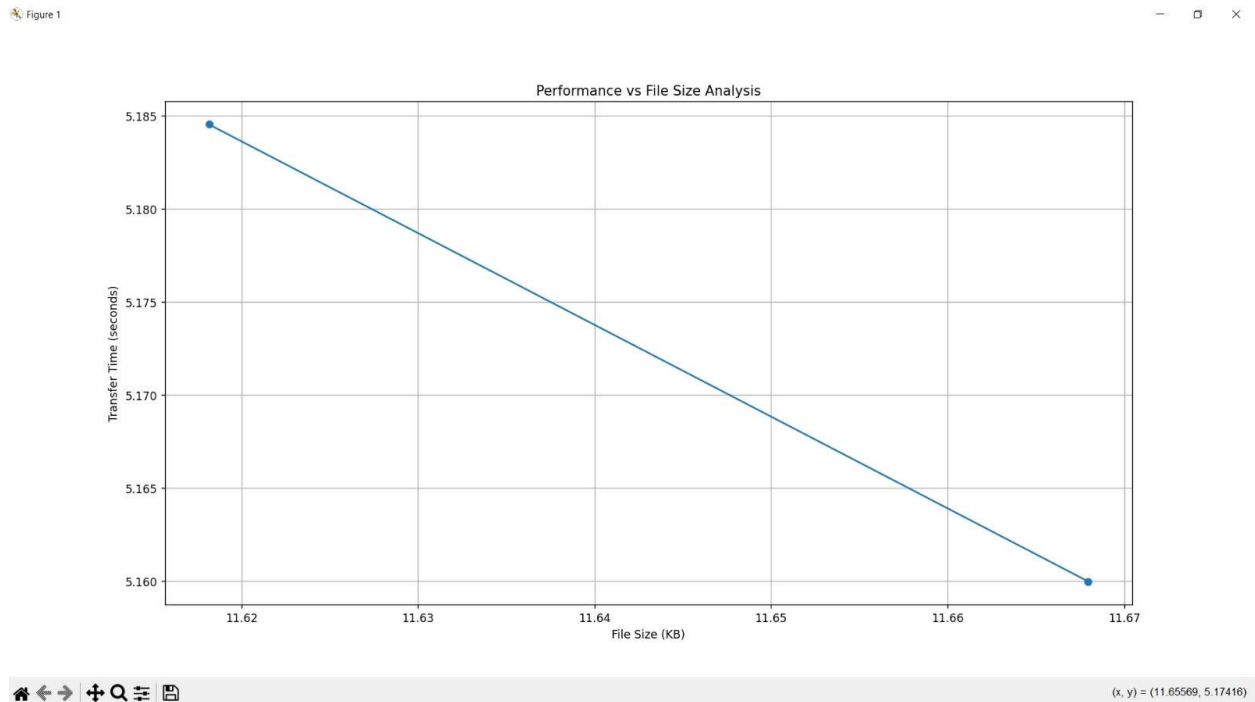
How it works

- Reads performance.csv (two columns: file size in bytes, time taken in seconds).
- Converts bytes to KB for the x-axis.

- Plots a line graph with matplotlib showing transfer time vs file size.

Things worth noting

- No error handling if performance.csv doesn't exist yet.
- `plt.show()` opens an interactive window — fine for local use, but you'd want `plt.savefig()` if running headless.



6. How the Files Connect

Here's the order things happen when you run the system:

1. You start `server.py` — it boots up 3 worker threads and starts listening.
2. You run the client and type a file path.
3. The client connects and sends the file over TCP.
4. `server.py` receives it and puts it in the job queue.
5. A worker thread picks it up and calls `conversion.start()`.
6. `conversion.py` converts the `.docx` to `.pdf` and returns the path.
7. The server sends the PDF back to the client.
8. The client saves it and writes the time taken to `performance.csv`.
9. Later you can run `performance_graph.py` to see a chart of results.

8. Summary

Overall, the system works well as a demonstration of socket programming, threading, and job queues in Python. The custom framing protocol is solid and handles TCP's stream nature correctly. The main weaknesses are around security (no encryption or auth) and file handling robustness, which would need to be addressed before using this in any real deployment.

9. Github Link:

https://github.com/Hamza-Sahapurwala/Sem_4_CN_Project_PESU